

فصل ۴

نخ‌ها و همروندی

فرض مدل پردازه‌ای که در فصل ۳ معرفی شد بر این بود که پردازه، برنامه‌ای در حال اجرا با یک نخ کنترل منفرد است. با این وجود، تقریباً تمامی سیستم‌عامل‌های مدرن امکاناتی فراهم می‌کنند که یک پردازه را قادر می‌سازد تا چندین نخ کنترل را در بر بگیرد. شناسایی فرصت‌های موازی‌سازی از طریق بهره‌گیری از نخ‌ها در سیستم‌های چندهسته‌ای مدرن که چندین CPU دارند به‌طور روزافزون اهمیت پیدا می‌کند.

در این فصل، به معرفی بسیاری از مفاهیم، همین‌طور چالش‌های مربوط به سیستم‌های کامپیوتری چندنخی، اعم از مباحث APIهای نخ‌های پوزیکس، ویندوز و کتابخانه‌های نخ جاوا خواهیم پرداخت. به‌علاوه، چندین امکان جدیدی را بررسی می‌کنیم که مفهوم ایجاد نخ‌ها را انتزاعی می‌سازند، به‌این ترتیب توسعه‌دهندگان اجازه می‌یابند تا بر شناسایی فرصت‌های موازی‌سازی تمرکز پیدا کنند و به‌ویژگی‌های زبان و چارچوب‌های API جزئیات ایجاد و مدیریت نخ فراهم می‌شود. نگاهی به برخی ملاحظات مربوط به برنامه‌نویسی چندنخی و تأثیر آن بر طراحی سیستم‌عامل‌ها می‌اندازیم. سرانجام، چگونگی پشتیبانی نخ‌ها در سطح هسته‌ی سیستم‌عامل‌های ویندوز و لینوکس را مرور می‌کنیم.

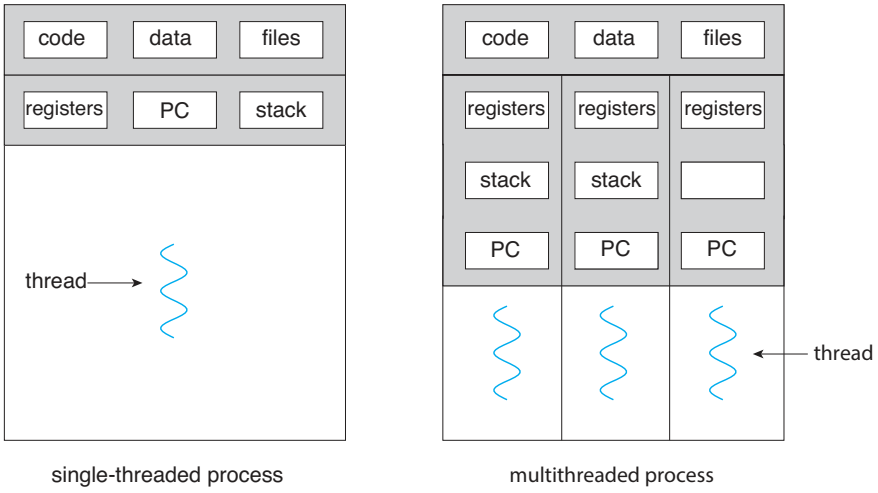
اهداف فصل

- شناسایی اجزای مبنایی یک نخ و مقایسه‌ی نخ‌ها و پردازه‌ها.
- تبیین منافع اصلی و چالش‌های مهم طراحی پردازه‌های چندنخی.
- پرداختن به رویکردهای متفاوت نخ‌سازی ضمنی، اعم از انبارهای نخ، fork-join و اعزام مرکزی بزرگ[1].
- تبیین چگونگی نمایش نخ‌های سیستم‌عامل‌های ویندوز و لینوکس.
- طراحی کاربردهای چندنخی با استفاده از API نخ‌سازی پوزیکس، جاوا، و ویندوز.

1. Grand Central Dispatch

۴-۱ کلیات

نخ به عنوان واحد اساسی بهره‌وری پردازنده است که مرکب از یک ID نخ، شمارنده‌ی برنامه (PC)، یک مجموعه ثبات و یک پشته می‌باشد. آن بخش کد، بخش داده‌ها و سایر منابع سیستم‌عامل خود را نظیر فایل‌های باز و سیگنال‌ها با سایر نخ‌های متعلق به پردازه‌ای که در آن قرار دارد، به اشتراک می‌گذارد. پردازه‌ی سابق، یک نخ کنترل منفرد داشت. اگر پردازه‌ای نخ‌های کنترل متعددی داشته باشد، می‌تواند در هر لحظه بیش از یک وظیفه را انجام دهد. شکل ۴-۱، فرق میان پردازه‌ی **تک‌نخی**[1] مرسوم و پردازه‌ی **چندنخی**[2] را روشن می‌کند.



شکل ۴-۱ پردازه‌های تک‌نخی و چندنخی.

۴-۱-۱ انگیزش

اکثر کاربردهای نرم‌افزاری که بر روی کامپیوترهای مدرن و دستگاه‌های موبایل اجرا می‌شوند، چندنخی هستند. هر برنامه‌ی کاربردی نوعاً به صورت پردازه‌ای مجزا با نخ‌های کنترل متعدد پیاده‌سازی می‌شود. در زیر، چند مثال از کاربردهای چندنخی را مطرح می‌کنیم:

- کاربردی که آلبومی از عکس‌ها را کوچک شده[3] نشان می‌دهد می‌تواند از یک نخ مجزایی برای تولید عکس کوچک هر یک از تصاویر جداگانه استفاده کند.
- یک مرورگر وب می‌تواند نخی برای نمایش تصاویر یا متن داشته باشد، در حالی که نخ دیگری هم برای دریافت داده‌ها از شبکه دارد.
- یک واژه‌پرداز می‌تواند نخی برای نمایش گرافیک، نخ دیگری برای پاسخ به کلیدهای ورودی کاربر و نخ سومی هم در پس‌زمینه برای بررسی درستی املاء و گرامر

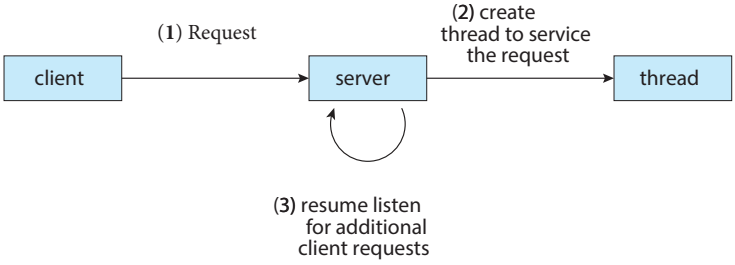
1. Single-threaded
2. Multithreaded
3. Thumbnail

داشته باشد.

برنامه‌های کاربردی می‌توانند طوری طراحی شوند که از قابلیت‌های سیستم‌های چندهسته‌ای نیز بهره ببرند. این قبیل برنامه‌های کاربردی می‌توانند چندین وظیفه پاپیچ پردازنده را به صورت موازی در هسته‌های محاسباتی متعدد به انجام برسانند.

در شرایط خاص ممکن است لازم شود که یک برنامه‌ی کاربردی منفرد بتواند چندین وظیفه‌ی مشابه را انجام بدهد. برای مثال، کارگزار وبی که درخواست کارفرماها را برای صفحات وب، تصاویر، صوت و از این دست می‌پذیرد. کارگزار وبی که مشغول سرویس‌دهی است چندین کارفرما (شاید هزاران) خواهد داشت که به آن دسترسی دارند. اگر کارگزار وب به شکل پردازه‌ی تک‌نخی سابق اجرا شود، در هر لحظه فقط می‌تواند به یک کارفرما سرویس دهد. لذا، مدت زمانی که هر کارفرما برای سرویس درخواست خود به ناچار منتظر می‌ماند بسیار بالا می‌رود.

یک راه‌حل این است که کارگزاری داشته باشیم که به صورت تک‌پردازه اجرا شده و درخواست‌ها را می‌پذیرد. زمانی که کارگزار درخواستی دریافت می‌کند، پردازه‌ی جداگانه‌ای جهت سرویس آن درخواست ایجاد می‌کند. در حقیقت، پیش از فراگیرشدن نخ‌ها، ایجاد پردازه یک روش رایج بود. همان‌گونه که در فصل قبل نشان داده شد، ایجاد پردازه وقت‌گیر و مصرف منبع بالایی دارد. اگر پردازه‌ی جدید همان وظایف پردازه‌ی موجود را انجام خواهد داد، چرا بایستی این همه سربار داشته باشیم؟ در حالت کلی بهتر است پردازه‌ای داشته باشیم که چندین نخ دارد. این روش، پردازه‌ی کارگزار وب را چندنخی خواهد ساخت. کارگزار، نخ مجزایی برای گوش‌ایستادن به درخواست کارفرماها ایجاد می‌کند. زمانی که درخواستی می‌رسد، کارگزار به جای ایجاد پردازه‌ای دیگر، نخ دیگری را برای سرویس درخواست و گوش‌ایستادن به درخواست‌های دیگر ایجاد خواهد کرد. این روش در شکل ۴-۲ به تصویر کشیده می‌شود.



شکل ۴-۲ معماری کارگزار چندنخی.

اکثر هسته‌های سیستم‌عامل‌های کنونی نیز نوعاً چندنخی هستند. به‌عنوان یک مثال، در طی زمان راه‌اندازی سیستم در سیستم‌های لینوکس، چندین نخ هسته ایجاد می‌شوند. هر نخ یک کار خاصی انجام می‌دهد، همچون مدیریت دستگاه‌ها، مدیریت حافظه، یا رسیدگی به وقفه. می‌توان از فرمان ps –ef برای نمایش نخ‌های هسته بر روی یک سیستم در حال اجرای لینوکس استفاده کرد. بررسی خروجی این فرمان، نخ kthread (با pid = 2) را نشان خواهد داد که به‌عنوان والد تمامی نخ‌های دیگر عمل می‌کند.

بسیاری از برنامه‌ها نیز می‌توانند از نخ‌های متعدد بهره‌مند شوند، اعم از الگوریتم‌های مرتب‌سازی مبنایی، درخت‌ها، و گراف. به‌علاوه، برنامه‌نویسانی که بایستی مسئله‌های کنونی پاپیچ CPU را در داده‌کاوی، گرافیک، و هوش مصنوعی حل کنند می‌توانند با طراحی جواب‌هایی که اجرای موازی دارند از قدرت سیستم‌های چندهسته‌ای مدرن بهره‌برداری کنند.

۲-۱-۴ منافع

منافع برنامه‌نویسی چندنخی را می‌توان به چهار دسته‌ی اصلی تقسیم کرد که عبارتند از:

۱- **پاسخگو بودن**: چندنخی کردن یک برنامه‌ی کاربردی محاوره‌ای باعث می‌شود تا برنامه حتی با وجود انسداد یا طولانی شدن عملیات بخشی از آن، همچنان به اجرای خود ادامه بدهد که این امر، پاسخگویی در قبال کاربر را بهبود می‌بخشد. این کیفیت مخصوصاً در طراحی رابط‌های کاربری مفید است. برای نمونه، در نظر بگیرید چه اتفاقی می‌افتد زمانی که کاربری دکمه‌ای را کلیک کند که منجر به انجام یک عملیات وقت‌گیر است. یک برنامه‌ی تک‌نخی، مادامی که عملیات تکمیل نشده، برای کاربر پاسخگو نخواهد بود. در مقایسه، اگر عملیات وقت‌گیر در نخ مجزای ناهمگام[1] انجام شود، برنامه برای کاربر پاسخگو می‌ماند.

۲- **اشتراک منبع**: پردازه‌ها می‌توانند اشتراک منابع را فقط از طریق تکنیک‌هایی چون حافظه‌ی مشترک یا عبور پیغام انجام بدهند. این قبیل تکنیک‌ها بایستی آشکارا توسط برنامه‌نویس جفت و جور شوند. با این وجود، نخ‌ها حافظه و منابع پردازه‌ای را که به آن تعلق دارند به طور پیش‌فرض به اشتراک می‌گیرند. مزیت اشتراک کد و داده‌هاست که داشتن چندین نخ فعال را در فضای آدرس یکسان برای یک برنامه‌ی کاربردی فراهم می‌کند.

۳- **صرفه‌ی اقتصادی**: تخصیص حافظه و منابع در ایجاد پردازه هزینه می‌برد. از این نظر که نخ‌ها از منابع پردازه‌ای که به آن تعلق دارند به طور مشترک استفاده می‌کنند، اقتصادی‌تر است که نخ‌هایی ایجاد کرده و بین آنها برگردان متن داشته باشیم. سنجش اختلاف سربار در عمل مشکل است؛ اما، در حالت کلی، ایجاد نخ نسبت به پردازه، زمان و حافظه‌ی کمتری مصرف می‌کند. به‌علاوه، برگردان متن میان نخ‌ها نیز نوعاً نسبت به پردازه‌ها سریع‌تر است.

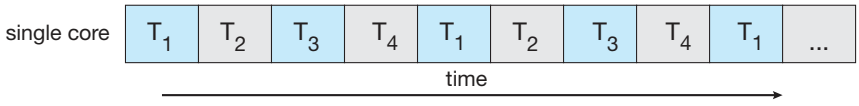
۴- **توسعه‌پذیری**[2]: منافع چندنخی می‌تواند در معماری چندپردازنده حتی عالی‌تر شود که در آن، نخ‌ها می‌توانند به طور موازی در هسته‌های پردازشی متفاوتی اجرا شوند. یک پردازه‌ی تک‌نخی، فارغ از تعداد پردازنده‌های موجود، فقط می‌تواند در یک پردازنده اجرا شود. این موضوع را در بخش بعدی بیشتر بررسی می‌کنیم.

۲-۴ برنامه‌نویسی چندهسته‌ای

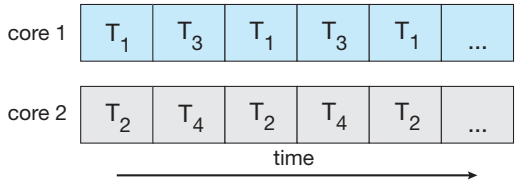
در اوایل تاریخچه‌ی طراحی کامپیوتر، در جواب نیاز کارایی محاسباتی بیشتر، سیستم‌های

1. Asynchronous
2. Scalability

تک‌پردازنده به سیستم‌های چندپردازنده تکامل پیدا کردند. یک گرایش اخیر مشابه در طراحی سیستم، گذاشتن هسته‌های[1] محاسباتی متعدد بر روی یک تراشه‌ی پردازشی منفرد است که هر هسته به‌عنوان پردازنده‌ای مجزا برای سیستم‌عامل محسوب می‌شود (بخش ۱-۳-۲). ما از این قبیل سیستم‌ها تحت عنوان **چندهسته‌ای** یاد می‌کنیم و برنامه‌نویسی چندنخی، راهکاری برای به‌کارگیری سودمند این هسته‌های محاسباتی متعدد و بهبود همروندی فراهم می‌کند. یک برنامه‌ی کاربردی را با چهار نخ در نظر بگیرید. همروندی در یک سیستم با یک هسته‌ی محاسباتی فقط به معنی اجرای یک در میان نخ‌ها در طول زمان خواهد بود (شکل ۴-۳)، چون هسته‌ی پردازشی، فقط قابلیت اجرای یک نخ را در هر لحظه دارد. با این وجود، همروندی در یک سیستم با هسته‌های متعدد به این معنی است که بعضی از نخ‌ها می‌توانند اجرای موازی داشته باشند، چون سیستم می‌تواند یک نخ مجزا را بر روی هر هسته بگمارد (شکل ۴-۴).



شکل ۴-۳ اجرای همروند بر روی یک سیستم تک‌هسته‌ای.



شکل ۴-۴ اجرای موازی بر روی یک سیستم چندهسته‌ای.

در این مبحث به تفاوت میان ***همروندی و موازی‌سازی*** دقت کنید. سیستم همروند با ایجاد امکان پیشروی برای تمامی وظایف، از اجرای بیش از یک وظیفه پشتیبانی می‌کند. در مقایسه، سیستم موازی می‌تواند بیش از یک وظیفه را به صورت همزمان انجام دهد. بنابراین، می‌توان بدون موازی‌سازی، همروندی داشت. قبل از ظهور معماری‌های چندپردازنده و چندهسته‌ای، اکثر سیستم‌های کامپیوتری فقط یک پردازنده داشتند، و زمانبندهای پردازنده چنان طراحی می‌شدند که تصور موازی‌سازی را با انتقال سریع میان پردازه‌های سیستم فراهم می‌کردند و با این کار، به هر یک از پردازه‌ها امکان پیشروی می‌دادند. این قبیل پردازه‌ها اجرای همروند داشتند، اما موازی نبودند.

همچنان که سیستم‌ها از ده‌ها نخ به هزاران نخ رشد پیدا کردند، طراحان پردازنده کارایی سیستم را با افزودن سخت‌افزاری برای بهبود کارایی نخ ارتقا دادند. پردازنده‌های مدرن اینتل در بیشتر موارد برای هر هسته از دو نخ پشتیبانی می‌کنند، در حالی که پردازنده‌ی T4 اوراکل به ازای هر هسته از هشت نخ پشتیبانی می‌کند. این پشتیبانی بدان معنی است که نخ‌های متعددی می‌توانند برای انتقال سریع به داخل هسته بار شوند. شکی

1. Cores

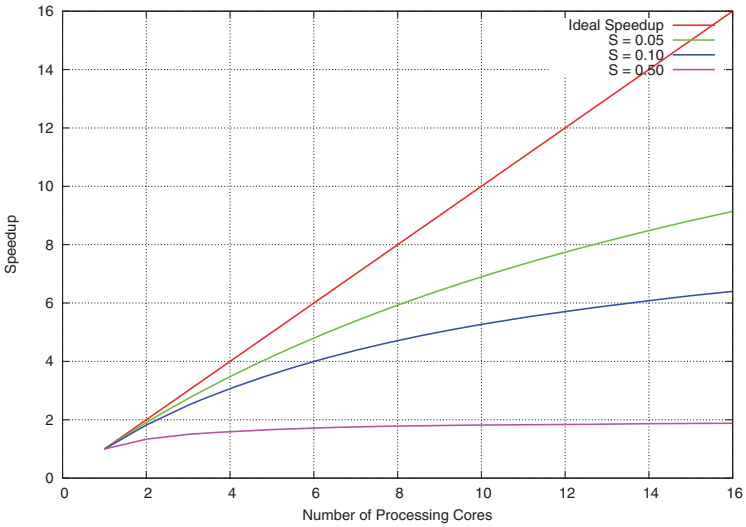
نیست که کامپیوترهای چندهسته‌ای در تعداد هسته‌ها و پشتیبانی سخت‌افزاری نخ به رشد خود ادامه خواهند داد.

قانون AMDAHL

قانون Amdahl فرمولی است که کارایی بالقوه‌ی حاصل از افزودن هسته‌های محاسباتی اضافه را برای یک برنامه‌ی کاربردی که دارای هر دو مؤلفه‌ی سری (غیرموازی) و موازی است، مشخص می‌کند. اگر S بخشی از برنامه باشد که بایستی روی یک سیستم با N هسته‌ی پردازشی به صورت سری انجام شود، فرمول آن به صورت زیر است:

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

به عنوان یک مثال، فرض کنید برنامه‌ای داریم که ۷۵ درصد آن موازی و ۲۵ درصد سری است. اگر این برنامه را روی یک سیستم با دو هسته‌ی پردازشی اجرا کنیم، تسریعی[1] از مرتبه‌ی ۱/۶ به‌دست می‌آوریم. اگر دو هسته‌ی دیگر اضافه کنیم (در مجموع با چهار هسته)، مرتبه‌ی تسریع ۲/۲۸ می‌شود. نمودار شکل زیر قانون Amdahl را در چندین سناریوی متفاوت نشان می‌دهد.



یک حقیقت جالب در رابطه با قانون Amdahl این است که با میل N به سمت بی‌نهایت، تسریع به $1/S$ همگرا می‌شود. برای مثال، اگر ۵۰ درصد از یک برنامه به صورت سری انجام شود، فارغ از تعداد هسته‌های پردازشی که اضافه کنیم، بیشینه‌ی تسریع ۲/۰ می‌شود. این به واقع اصل مبنایی پشت سر قانون Amdahl است: بخش سری یک برنامه می‌تواند اثر نامتناسبی روی کارایی داشته باشد که با افزودن هسته‌های محاسباتی اضافی عاید ما می‌شود.

1. Speedup

برخی بر این باورند که قانون Amdahl بهبودهای کارایی سخت‌افزاری صورت گرفته در طراحی سیستم‌های چندهسته‌ای فعلی را نادیده می‌گیرد. پیشنهاد این قبیل استدلال‌ها این است که با تداوم افزایش هسته‌های پردازشی بر روی سیستم‌های کامپیوتری مدرن ممکن است کاربرد قانون Amdahl را متوقف کنیم.

۴-۲-۱ چالش‌های برنامه‌نویسی

گرایش به سمت سیستم‌های چندهسته‌ای، پیوسته طراحان سیستم را در کنار برنامه‌نویسان کاربردی تحت فشار قرار می‌دهد تا استفاده‌ی بهتری از هسته‌های محاسباتی متعدد داشته باشند. طراحان سیستم‌عامل‌ها بایستی الگوریتم‌های زمانبندی را چنان بنویسند که با بهره‌گیری از هسته‌های محاسباتی متعدد، امکان اجرای موازی نشان داده شده در شکل ۴-۴ فراهم شود. چالش برنامه‌نویسان کاربردی، اصلاح برنامه‌های موجود در کنار طراحی برنامه‌های جدیدی است که چندنخی باشند.

در حالت کلی، چالش برنامه‌نویسان سیستم‌های چندهسته‌ای در پنج بخش متمرکز است که عبارتند از:

۱- **شناسایی وظیفه‌ها**: این مورد شامل بررسی برنامه‌های کاربردی است تا بتوان بخش‌هایی را یافت که به وظایف مجزا و همروند تقسیم می‌شوند. به صورت ایده‌آل، وظیفه‌ها مستقل از یکدیگرند و از این رو می‌توانند به صورت موازی روی تک‌تک هسته‌ها اجرا شوند.

۲- **تعادل**: برنامه‌نویسان در حالی که وظایفی را برای اجرای موازی شناسایی می‌کنند، بایستی مطمئن شوند که وظیفه‌ها، کاری مساوی، با ارزش برابر انجام می‌دهند. در شرایطی ممکن است یک وظیفه‌ی خاص، مشارکت زیادی بر روی کل پردازه، مثل سایر وظایف نداشته باشد. ممکن است به‌کارگیری یک هسته‌ی اجرایی مجزا برای اجرای چنین وظیفه‌ای ارزش هزینه‌اش را نداشته باشد.

۳- **شکستن داده‌ها**: همچنان که برنامه‌های کاربردی به وظایف مجزایی تقسیم می‌شوند، دسترسی و دستکاری داده‌ها توسط وظیفه‌ها هم بایستی تقسیم شوند تا روی هسته‌های مجزا قابلیت اجرا داشته باشند.

۴- **وابستگی داده‌ها**: وابستگی داده‌هایی که توسط وظیفه‌ها دسترسی می‌شوند بایستی بررسی شود. موقعی که یک وظیفه به وظیفه‌ای دیگر وابستگی داده دارد، برنامه‌نویسان می‌بایست به این اطمینان برسند که اجرای وظیفه‌ها جهت پوشش وابستگی داده‌هایشان همگام می‌شود. این راهبردها را در فصل ۶ بررسی می‌کنیم.

۵- **آزمون و عیب‌یابی**: زمانی که یک برنامه به صورت موازی روی چندین هسته اجرا می‌شود، مسیرهای اجرایی زیادی امکان‌پذیرند. آزمون و عیب‌یابی این قبیل برنامه‌های همروند ذاتاً مشکل‌تر از آزمون و عیب‌یابی برنامه‌های تک‌نخی خواهد بود.

به خاطر این چالش‌های موجود، بسیاری از تولیدکنندگان نرم‌افزار معتقدند که پیدایش

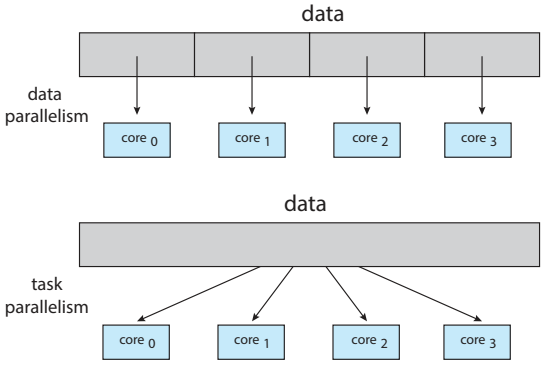
سیستم‌های چندهسته‌ای، رویکرد کاملاً جدیدی را در طراحی سیستم‌های نرم‌افزاری آینده لازم خواهد داشت. (به طور مشابه، بسیاری از مربیان علوم کامپیوتر معتقدند که توسعه‌ی نرم‌افزار بایستی با تأکید بیشتر بر روی برنامه‌نویسی موازی تدریس شود.)

۴-۲-۲ انواع موازی‌سازی

در حالت کلی، دو نوع موازی‌سازی وجود دارد: موازی‌سازی داده‌ها و موازی‌سازی وظیفه. تمرکز **موازی‌سازی داده‌ها** بر توزیع زیرمجموعه‌های داده‌های یکسان در هسته‌های محاسباتی متعدد و انجام عملیات یکسان بر روی هر یک از هسته‌ها می‌باشد. به عنوان مثال، جمع‌کردن محتوای یک آرایه با اندازه‌ی N را در نظر بگیرید. در سیستم تک‌هسته‌ای، یک نخ به سادگی عناصر [0] تا [N – 1] را جمع خواهد کرد. با این وجود، در سیستم دوهسته‌ای، نخ A که روی هسته‌ی ۰ اجرا دارد می‌تواند عناصر [0] تا [N/2 – 1] را جمع کند در حالی که نخ B که روی هسته‌ی ۱ اجرا می‌شود، می‌تواند عناصر [N/2] تا [N – 1] را جمع کند. دو نخ در هسته‌های محاسباتی جداگانه اجرای موازی خواهند داشت.

موازی‌سازی وظیفه مشمول توزیع داده‌ها نیست، بلکه در اینجا وظیفه‌ها (نخ‌ها) در هسته‌های محاسباتی متعدد توزیع می‌شوند. هر نخ عملیات یکتایی را انجام می‌دهد. نخ‌های متفاوت ممکن است روی داده‌های یکسان، یا روی داده‌های متفاوت عملیات داشته باشند. باز هم مثال بالا را در نظر بگیرید. در مقایسه با شرایط مذکور، یک مثال از موازی‌سازی وظیفه می‌تواند شامل دو نخ باشد که هر کدام مشغول انجام یک عملیات آماری بر روی آرایه‌ی عناصر هستند. باز هم نخ‌ها به صورت موازی بر روی هسته‌های محاسباتی جداگانه عملیات دارند، اما هر کدام یک عملیات یکتا انجام می‌دهند.

پس، موازی‌سازی داده‌ها در اصل شامل توزیع دادها در هسته‌های متعدد بوده و موازی‌سازی وظیفه مشمول توزیع وظیفه‌ها در هسته‌های متعدد است، آن‌گونه که در شکل ۴-۵ نشان داده می‌شود. با این همه، موازی‌سازی داده‌ها و وظیفه انحصار متقابل نداشته، و یک برنامه می‌تواند به‌واقع ترکیبی از این دو راهبرد را به‌کار ببرد.



شکل ۴-۵ موازی‌سازی داده‌ها و وظیفه.